

Esame Programmazione di Sistema / API Programming

Data: 2021-09-02

THEORETICAL AND PROGRAMMING QUESTIONS

In un sistema concorrente sono presenti due gruppi di thread, detti rispettivamente produttori e consumatori, che si appoggiano ad una struttura dati condivisa per passarsi dati generici di tipo `T`. La struttura dati è *thread-safe* ed implementa il concetto di buffer circolare: al suo interno ospita un array di `N` elementi (con `N` specificato a livello di tipo) nel quale vengono depositati i valori ricevuti dai singoli produttori, in attesa che vengano passati ad un consumatore qualunque che ne faccia richiesta. I dati sono trattati in modalità FIFO (First-In-First-Out).

- `pub fn insert(&self, t: T) {...}` → Inserisce un elemento. Se il buffer risulta pieno nel momento in cui un produttore prova ad inserire un nuovo valore, l'operazione si blocca, senza consumare CPU, in attesa che si crei uno spazio a seguito della conclusione di un'operazione di lettura da parte di un consumatore.
- `pub fn extract(&self) -> T {...}` → Estrae un elemento. Analogamente, se un consumatore prova a leggere un valore mentre il buffer è vuoto, deve attendere, senza consumare CPU, che un produttore inserisca un nuovo valore.

Per generalità, accanto alle operazioni base di inserimento ed estrazione di un valore, la struttura dati offre anche una coppia di operazioni con attesa limitata temporalmente:

- `pub fn try_insert_for(&self, t: T, d: Duration) -> TimeoutResult {...}` → Cerca di inserire un elemento, se non riesce entro l'intervallo `duration`, restituisce `false`; in caso di successo, restituisce `true`.
- `pub fn try_extract_for(&self, d: Duration) -> (Option<T>, TimeoutResult) {...}` → Cerca di estrarre un elemento, se non riesce entro l'intervallo `duration` restituisce un `Option` vuoto; altrimenti restituisce un `Option` con l'elemento estratto.

Si implementi la classe generica di seguito proposta utilizzando le funzionalità offerte dal linguaggio **Rust**, aggiungendo le parti eventualmente necessarie per ottenere il comportamento descritto.

```
1 type TimeoutResult = bool;
2
3 struct CircularBuffer<T: Send + 'static> {
4     // ...
5 }
6
7 impl<T: Send + 'static> CircularBuffer<T> {
8     // ...
9 }
```

ANSWERS

Il problema richiede l'implementazione di un buffer circolare *thread-safe* di dimensione finita N. Essendo N "specificato a livello di tipo", dovremo estendere la firma del tipo per includere un *Const Generic* `const N: usize`, funzionalità introdotta a partire da Rust 1.51.

Per gestire la sincronizzazione in modo efficiente ed evitare consumi di CPU inutili (busy waiting), faremo uso di un `Mutex` per proteggere lo stato interno e di due `Condvar` (Condition Variables) per orchestrare l'attesa coordinata di produttori e consumatori.

Lo stato interno del buffer sarà così composto:

- Un array `[Option<T>; N]` per ospitare gli elementi in modo contiguo senza locazioni dinamiche.
- `head`: indice circolare dal quale avviene l'estrazione.
- `tail`: indice circolare nel quale avviene l'inserimento.
- `count`: contatore per tracciare il numero attuale di elementi nel buffer e discriminare tra stato "pieno" e "vuoto".

Di seguito è riportata la soluzione completa in Rust:

```
1 use std::sync::{Condvar, Mutex};
2 use std::time::{Duration, Instant};
3
4 pub type TimeoutResult = bool;
5
6 // Struttura che incapsula lo stato condiviso
7 struct State<T, const N: usize> {
8     buffer: [Option<T>; N],
9     head: usize,
10    tail: usize,
11    count: usize,
12 }
13
14 pub struct CircularBuffer<T: Send + 'static, const N: usize> {
15     state: Mutex<State<T, N>>,
16     not_empty: Condvar,
17     not_full: Condvar,
18 }
19
20 impl<T: Send + 'static, const N: usize> CircularBuffer<T, N> {
21     // Trucco per inizializzare un array di elementi non-Copy con None
22     const NONE: Option<T> = None;
23
24     pub fn new() -> Self {
25         Self {
26             state: Mutex::new(State {
27                 buffer: [Self::NONE; N],
28                 head: 0,
29                 tail: 0,
30                 count: 0,
31             }),
32             not_empty: Condvar::new(),
33             not_full: Condvar::new(),
34         }
35     }
36 }
```

```

37 pub fn insert(&self, t: T) {
38     let mut state = self.state.lock().unwrap();
39
40     // Attendiamo finche' il buffer e' pieno (evitiamo spurious
41     // wakeups)
42     while state.count == N {
43         state = self.not_full.wait(state).unwrap();
44     }
45
46     state.buffer[state.tail] = Some(t);
47     state.tail = (state.tail + 1) % N;
48     state.count += 1;
49
50     // Notifichiamo un thread in attesa su estrazione
51     self.not_empty.notify_one();
52 }
53
54 pub fn extract(&self) -> T {
55     let mut state = self.state.lock().unwrap();
56
57     // Attendiamo finche' il buffer e' vuoto
58     while state.count == 0 {
59         state = self.not_empty.wait(state).unwrap();
60     }
61
62     let t = state.buffer[state.head].take().unwrap();
63     state.head = (state.head + 1) % N;
64     state.count -= 1;
65
66     // Notifichiamo un thread in attesa su inserimento
67     self.not_full.notify_one();
68
69     t
70 }
71
72 pub fn try_insert_for(&self, t: T, d: Duration) -> TimeoutResult {
73     let mut state = self.state.lock().unwrap();
74     let mut remaining = d;
75     let mut now = Instant::now();
76
77     while state.count == N {
78         let (new_state, timeout_res) = self.not_full.wait_timeout(
79             state, remaining).unwrap();
80         state = new_state;
81
82         if timeout_res.timed_out() {
83             return false; // Tempo scaduto, inserimento fallito
84         }
85
86         let elapsed = now.elapsed();
87         if elapsed >= remaining {
88             return false;
89         }
90
91         remaining -= elapsed;
92         now = Instant::now();
93     }
94 }

```

```

93     state.buffer[state.tail] = Some(t);
94     state.tail = (state.tail + 1) % N;
95     state.count += 1;
96
97     self.not_empty.notify_one();
98     true
99 }
100
101 pub fn try_extract_for(&self, d: Duration) -> (Option<T>,
TimeoutResult) {
102     let mut state = self.state.lock().unwrap();
103     let mut remaining = d;
104     let mut now = Instant::now();
105
106     while state.count == 0 {
107         let (new_state, timeout_res) = self.not_empty.wait_timeout(
state, remaining).unwrap();
108         state = new_state;
109
110         if timeout_res.timed_out() {
111             return (None, false); // Tempo scaduto, nessuna
estrazione
112         }
113
114         let elapsed = now.elapsed();
115         if elapsed >= remaining {
116             return (None, false);
117         }
118
119         remaining -= elapsed;
120         now = Instant::now();
121     }
122
123     let t = state.buffer[state.head].take().unwrap();
124     state.head = (state.head + 1) % N;
125     state.count -= 1;
126
127     self.not_full.notify_one();
128     (Some(t), true)
129 }
130 }

```

Dettagli Implementativi

- **Const Generics per il buffer:** Modificando la firma in `<T, const N: usize>` possiamo sfruttare a compile-time la dimensione statica del buffer. Per aggirare il vincolo Copy necessario per la macro d’inizializzazione array `[...; N]`, si definisce la costante `NONE` che permette di istanziare validamente un array fisso `[Self::NONE; N]`.
- **Condvar Wait Loop:** Sia in estrazione che in inserimento, la chiamata a `wait()` viene circonscritta in un loop `while` sulla condizione di interesse (`state.count == 0` oppure `state.count == N`) per ovviare alla problematica dei *spurious wakeups* supportati nativamente dalle librerie POSIX.
- **Gestione Attese Temporizzate:** I metodi `try_*` utilizzano la funzione `wait_timeout` e ricalcolano ad ogni ciclo il tempo residuo. In questo modo si mantengono sicuri ed efficaci anche a fronte di risvegli imprevisti, garantendo che non eccedano il tempo limite `d`.